

5.4. Introduction to Level 2 View

{#level-2-view}

This section delves deeper into the architecture by describing the key data structures within the system. These are represented by Django models, which define the schema for the database and encapsulate domain-specific data and behavior. We will focus on the most central models from key applications.

(Note: For brevity, not all fields of each model are listed, but the most structurally or functionally important ones are highlighted. `"EncryptedCharField/TextField"` implies data is encrypted at rest, typically via a library like `django-cryptography` or `django-fernet-fields` which was discussed during model implementation.)

5.4.1. User Model (`users.User`)

{#_5_4_1_user_model_users_user}

The `User` model represents any individual interacting with the system and forms the basis for authentication and role-based access.

App: `users`

Purpose: Stores core identity, authentication credentials, and a `user_type` to distinguish between Jobseekers, Publishers, and Support Users. It also consolidates common fields previously spread across separate `Jobseeker` and `Publisher` models in the Rails application for role-specific PII and identifiers.

Key Fields & Relationships: * `id`: UUIDField (Primary Key) * `email`: EmailField (Unique, Used as `USERNAME_FIELD`) * `password`: CharField (Hashed, managed by Django's auth system) * `user_type`: CharField (Choices: `UserType.JOBSEEKER`, `UserType.PUBLISHER`, `UserType.SUPPORT_USER`) * `is_active`: BooleanField (Controls if the user account is active) * `is_staff`: BooleanField (Grants access to Django admin interface) * `is_superuser`: BooleanField (Grants all permissions) * `date_joined`: DateTimeField (Timestamp of account creation) * `last_login_at`: DateTimeField (Timestamp of last login) * `govuk_one_login_id`: CharField (Unique, nullable, for Jobseekers using GOV.UK One Login) * `account_closed_on`: DateField (nullable, for Jobseekers) * `email_opt_out`: BooleanField (default=False, for Jobseekers) * `email_opt_out_reason`: IntegerField (nullable, choices from `EmailOptOutReason` enum, for Jobseekers) * `dfe_signin_oid`: CharField (Unique, nullable, for Publishers using DfE Sign-In) * `accepted_terms_at`: DateTimeField (nullable, for Publishers) * `family_name`: EncryptedCharField (nullable, for Publishers) - Data encrypted at rest * `given_name`: EncryptedCharField (nullable, for Publishers) - Data encrypted at rest * `dfe_signin_oid_support`: CharField (Unique, nullable, for Support Users using DfE Sign-In) * `support_given_name`: CharField (nullable, for Support Users) * `support_family_name`: CharField (nullable, for Support Users) * `groups`: ManyToManyField to `django.contrib.auth.models.Group` (Standard Django permissions) * `user_permissions`: ManyToManyField to `django.contrib.auth.models.Permission` (Standard Django permissions)

Notable Logic: * Custom `UserManager` with methods like `create_user()`, `create_superuser()`, `create_jobseeker()`, `create_publisher()` to handle role-specific user creation. * Email is used as the primary identifier for login (`USERNAME_FIELD = 'email'`).

5.4.2. Organisation Model

(organisations.Organisation)

{#_5_4_2_organisation_model_organisations_organisation}

The **Organisation** model represents an educational institution or group of institutions, such as a school, academy trust, or local authority.

App: **organisations**

Purpose: Stores profile information, address details, type, and administrative data for organisations that publish vacancies.

Key Fields & Relationships: * **id**: UUIDField (Primary Key) * **name**: CharField (Indexed) * **slug**: SlugField (Unique, auto-generated) * **description**: TextField (nullable) * **organisation_type**: CharField (Choices: OrganisationType.SCHOOL, OrganisationType.SCHOOL_GROUP) * **phase**: IntegerField (Choices: SchoolPhase enum, nullable) * **url**: URLField (nullable) * **email**: EmailField (nullable) * **address_line1**, **address_line2**, **town**, **county**, **postcode**: CharFields for address components * **geopoint_placeholder**: CharField (Placeholder for actual **PointField** which requires GDAL) * **urn**, **uid**, **local_authority_code**: CharFields (Unique, nullable, indexed, for external identifiers) * **group_type**: CharField (e.g., "Multi-academy trust", nullable) * **gias_data**: JSONField (nullable, stores raw data from GIAS) * **searchable_content**: SearchVectorField (nullable, for PostgreSQL full-text search) * **logo**: ImageField (nullable) * **photo**: ImageField (nullable) * **publishers**: ManyToManyField to **users.User** (Users who can manage this organisation, limited to **user_type** PUBLISHER) * **created_at**, **updated_at**: DateTimeFields

Notable Logic: * Automatic **slug** generation on save if not provided. * **searchable_content** is intended to be updated via signals or background tasks based on changes to relevant fields (name, postcode, town, etc.) for efficient full-text search. (Initial implementation in **save()** was noted as inefficient and to be refactored). * **geopoint_placeholder** is used temporarily; a real **PointField** would allow for geospatial queries.

5.4.3. Vacancy Model (`vacancies.Vacancy`)

{#_5_4_3_vacancy_model_vacancies_vacancy}

The `Vacancy` model is central to the platform, representing a single job advertisement.

App: `vacancies`

Purpose: Stores all details related to a job vacancy, including its description, salary, contract type, working patterns, application process, and publication status.

Key Fields & Relationships: * `id`: UUIDField (Primary Key) * `job_title`: CharField (Indexed) * `slug`: SlugField (Unique, auto-generated) * `job_advert`: TextField * `salary`: CharField (nullable) * `status`: CharField (Choices: `VacancyStatus` enum, default: `DRAFT`, indexed) * `expires_at`: DateTimeField (Indexed) * `publish_on`: DateField (Indexed) * `working_patterns`: ArrayField(CharField) (nullable, choices from `WorkingPattern` enum) * `contract_type`: CharField (Choices: `ContractType` enum, nullable) * `phases`: ArrayField(CharField) (nullable, choices from `PhaseChoice` enum) * `subjects`: ArrayField(CharField) (nullable) * `application_link`: URLField (nullable) * `enable_job_applications`: BooleanField (True if applications are via TVS) * `geopoint_placeholder`: CharField (Placeholder for actual `PointField`) * `searchable_content`: SearchVectorField (nullable) * `publisher`: ForeignKey to `users.User` (The publisher who created the vacancy, limited to `user_type` `PUBLISHER`, nullable) * `publisher_organisation`: ForeignKey to `organisations.Organisation` (The primary organisation publishing the vacancy, nullable) * `organisations`: ManyToManyField to `organisations.Organisation` (through `OrganisationVacancy`, for jobs at multiple schools in a group) * `discarded_at`: DateTimeField (nullable, for soft deletion) * `created_at`, `updated_at`: DateTimeFields

Notable Logic: * Automatic `slug` generation based on `job_title` on save. * `searchable_content` intended for update via signals/tasks. * Forward reference to `publishers.PublisherAtsApiClient` (as the `publishers` app and this model are not yet created). * Various enums define choices for status, contract type, working patterns, etc.

5.4.4. OrganisationVacancy Model

(`vacancies.OrganisationVacancy`)

`{#_5_4_4_organisationvacancy_model_vacancies_organisationvacancy}`

This is a through model managing the many-to-many relationship between Vacancies and Organisations.

App: `vacancies`

Purpose: Allows a single vacancy to be associated with multiple organisations, typically schools within a trust or local authority.

Key Fields & Relationships: `* id: UUIDField (Primary Key) * vacancy: ForeignKey to vacancies.Vacancy * organisation: ForeignKey to organisations.Organisation * created_at, updated_at: DateTimeFields`

Notable Logic: `* unique_together = ('vacancy', 'organisation')` constraint.

5.4.5. JobApplication Model

(`job_applications.JobApplication`)

{#_5_4_5_jobapplication_model_job_applications_jobapplication}

The `JobApplication` model stores the data submitted by a jobseeker for a specific vacancy.

App: `job_applications` (This app will be created in a subsequent step)

Purpose: To capture all information related to a jobseeker's application for a vacancy, including their personal details at the time of application, professional status, qualifications, employment history, statements, references, and equal opportunities data.

Key Fields & Relationships: * `id`: UUIDField (Primary Key) * `jobseeker`: ForeignKey to `users.User` (The applicant, limited to `user_type` JOBSEEKER) * `vacancy`: ForeignKey to `vacancies.Vacancy` * `status`: IntegerField (Choices: `JobApplicationStatus` enum, e.g., DRAFT, SUBMITTED, REVIEWED, SHORTLISTED, UNSUCCESSFUL, WITHDRAWN) * `completed_steps`: ArrayField(CharField) (Tracks completed sections of the application form) * `submitted_at`: DateTimeField (nullable) * `first_name`, `last_name`, `email_address`, `phone_number`, `street_address`, `city`, `postcode`: EncryptedCharFields/TextFields (PII related to the application) * `qualified_teacher_status`: CharField (nullable) * `personal_statement`: EncryptedTextField (nullable) * `created_at`, `updated_at`: DateTimeFields * (Many other fields for professional status, qualifications, employment history, references, equal opportunities data, etc. will be linked via OneToMany relationships to separate models like `Employment`, `Qualification`, `Referee`.)

Notable Logic: * Status management with associated timestamps (e.g., `submitted_at`, `reviewed_at`). * Anonymisation of equal opportunities data upon submission. * Logic to determine if an application is editable based on status and deadline.

5.4.6. JobseekerProfile Model

(`jobseekers.JobseekerProfile`)

`{#_5_4_6_jobseekerprofile_model_jobseekers_jobseekerprofile}`

The `JobseekerProfile` model stores reusable information about a jobseeker that they can use to quickly apply for jobs.

App: `jobseekers` (This app will be created in a subsequent step)

Purpose: Allows jobseekers to store their personal details, job preferences, qualifications, and work history independently of any single application, enabling faster applications and profile visibility to recruiters.

Key Fields & Relationships: * `jobseeker`: OneToOneField to `users.User` (Primary Key, limited to `user_type` JOBSEEKER) * `about_you`: TextField (nullable) * `qualified_teacher_status`: IntegerField (Choices: QTSChoices enum, nullable) * `teacher_reference_number`: EncryptedCharField (nullable) * `active`: BooleanField (Indicates if the profile is visible to publishers, default=False) * `personal_details`: OneToOneField to `jobseekers.PersonalDetails` (nullable) * `job_preferences`: OneToOneField to `jobseekers.JobPreference` (nullable) * `employments`: OneToMany from `job_applications.Employment` (shared model) * `qualifications`: OneToMany from `job_applications.Qualification` (shared model) * `created_at, updated_at`: DateTimeFields

Notable Logic: * `active` status controls visibility to publishers. * Logic to determine if a profile is "activable" based on completion of key sections. * Methods to replace collections of associated records (e.g., `replace_qualifications!`).

5.4.7. Subscription Model

(`jobseekers.Subscription`)

`{#_5_4_7_subscription_model_jobseekers_subscription}`

The `Subscription` model, also known as Job Alerts, stores jobseeker preferences for receiving notifications about new vacancies.

App: `jobseekers` (This app will be created in a subsequent step)

Purpose: Enables jobseekers to save search criteria and receive email notifications (daily or weekly) about new vacancies matching those criteria.

Key Fields & Relationships: `* id`: UUIDField (Primary Key) `* email`: EmailField (Indexed, can be different from a registered user's email if for anonymous alerts) `* frequency`: IntegerField (Choices: SubscriptionFrequency enum, e.g., DAILY, WEEKLY) `* search_criteria`: JSONField (Stores the search parameters for the alert) `* active`: BooleanField (default=True, indexed) `* unsubscribed_at`: DateTimeField (nullable) `* created_at, updated_at`: DateTimeFields

Notable Logic: `* Token generation/verification for unsubscribe links (likely handled in views/services).` `* Logic for matching vacancies to search_criteria (typically in a service or background task).`

5.4.8. Feedback Model (`core.Feedback`)

{#_5_4_8_feedback_model_core_feedback}

The `Feedback` model is a generic store for various types of user feedback provided on the platform.

App: `core`

Purpose: To collect and store feedback from users (jobseekers, publishers, general visitors) regarding their experience with the service, specific vacancies, job applications, or job alerts.

Key Fields & Relationships: * `id`: UUIDField (Primary Key) * `feedback_type`: CharField (Choices: `FeedbackType` enum, e.g., `GENERAL`, `JOB_ALERT`, `VACANCY_PUBLISHER`) * `rating`: IntegerField (Choices: `FeedbackRating` enum, nullable) * `comment`: TextField (nullable) * `email`: EmailField (nullable, if provided by an unauthenticated user) * `user`: ForeignKey to `users.User` (nullable, if the user is authenticated) * `vacancy`: ForeignKey to `vacancies.Vacancy` (nullable, if feedback relates to a vacancy) * `job_application`: ForeignKey to `job_applications.JobApplication` (nullable, if feedback relates to an application) * `subscription`: ForeignKey to `jobseekers.Subscription` (nullable, if feedback relates to a job alert) * `created_at`, `updated_at`: DateTimeFields

Notable Logic: * Polymorphic-like association to different entities (`User`, `Vacancy`, `JobApplication`, `Subscription`) via nullable ForeignKeys. * Various enums define the type, rating, and purpose of the feedback.